TRƯỜNG ĐẠI HỌC ……

KHOA ……

BÁO CÁO THỰC HÀNH

Môn học: Kỹ thuật dữ liệu (Data Engineering)

Bài thực hành L06

Tối ưu hiệu năng Spark: shuffle, chiến lược join, data skew, AQE, phân vùng và cache

Giảng viên hướng dẫn: ……

Sinh viên thực hiện: Trương Minh Tiến

Mã số sinh viên: ……

Lớp: ……

……, Tháng 10 năm 2026

Mục lục

Tóm tắt **3**

1 Giới thiệu **3**

2 Cơ sở lý thuyết **4**
2.1 Shuffle và số partition shuffle 4
2.2 Adaptive Query Execution 4
2.3 Chiến lược join 4
2.4 Data skew và salting 4
2.5 repartition, coalesce và cache 4

3 Môi trường và phương pháp thực nghiệm **5**
3.1 Môi trường 5
3.2 Dữ liệu 5
3.3 Quy trình 6

4 Kết quả thực nghiệm **6**
4.1 Tổng quan hai lần chạy 6
4.2 E1: số partition shuffle và AQE coalescing 11
4.3 E2: broadcast hash join và sort-merge join 12
4.4 E3: data skew, salting và AQE skew join 15
4.5 E4: repartition và coalesce trước khi ghi 20
4.6 E5: cache 20
4.7 So sánh với kết quả tham chiếu 21

5 Thảo luận **22**
5.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh? 22
5.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết? 22
5.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng? 22
5.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu? 23
5.5 Khi nào không nên broadcast dù một phía "nhỏ"? 23
5.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước? 23
5.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không? . 24
5.8 Nên tinh chỉnh gì trước khi thêm phần cứng? 24
5.9 Hạn chế của thực nghiệm 24

6 Kết luận **24**

Tài liệu tham khảo **25**

Phụ lục A: Danh mục tệp nộp kèm **26**

Phụ lục B: Trích đoạn mã nguồn **26**

Danh mục hình

1 Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể. 7
2 Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai. 7
3 Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy. 10
4 Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle. 11
5 Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB. 12
6 Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join. 13
7 Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`. 14
8 Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API. 15
9 Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch. 16
10 Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s. 17
11 Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms. 18
12 Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`. 19
13 Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task. 20
14 Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ. 21

Danh mục bảng

1 Môi trường thực nghiệm. 5
2 Dữ liệu thực nghiệm. 5
3 Kết quả đầy đủ của lần chạy thứ hai (17 biến thể). 7
4 So sánh một số chỉ số giữa hai lần chạy. 10
5 Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai). 15
6 So sánh với bảng kết quả tham chiếu của bài lab. 21

7 Danh mục tệp nộp kèm. 26

Tóm tắt

Bài này đo ảnh hưởng của một số tham số tinh chỉnh Apache Spark 4.1.3 trên một máy, chế độ `local[4]`. Dữ liệu gồm tập chuyến taxi New York tháng 01/2024 (2,964,624 dòng) và một bảng fact tổng hợp 10 triệu dòng, trong đó 60 % số dòng có cùng một khóa. Mỗi biến thể được ép chạy hết qua *noop sink*; thời gian đo bằng đồng hồ tường, kế hoạch vật lý cuối và thống kê task lấy từ Spark REST API. Với dữ liệu nhỏ, 200 partition shuffle chậm hơn khoảng 6.7 lần so với 8 partition, còn *Adaptive Query Execution* (AQE) tự gộp partition và về gần mức tinh chỉnh tay. Trong phép join bị lệch, task chậm nhất lâu hơn trung vị khoảng 14 lần; AQE skew join rút thời gian từ 4.10 s xuống 2.22 s, trong khi salting chỉ cắt được task chậm nhất mà thời gian tổng gần như giữ nguyên. Cache một DataFrame đắt được dùng lại ba lần giảm thời gian ba truy vấn từ 6.40 s xuống 0.99 s. Cuối bài là một số khuyến nghị cho job production trên cluster 20 node.

Từ khóa: Apache Spark, shuffle partitions, Adaptive Query Execution, broadcast hash join, sort-merge join, data skew, salting, repartition, coalesce, cache.

1 Giới thiệu

Tốc độ một job Spark phụ thuộc nhiều vào cách dữ liệu được chia cho các task: có bao nhiêu partition sau shuffle, planner chọn kiểu join nào, khóa có bị lệch không, ghi ra bao nhiêu file, và kết quả trung gian có được dùng lại không. Những lựa chọn này hay được đặt theo thói quen, hoặc khi chậm thì người ta thêm máy. Bài lab L06 yêu cầu đo thay vì đoán.

Năm thí nghiệm được thực hiện:

1. E1: ảnh hưởng của `spark.sql.shuffle.partitions` và cơ chế gộp partition của AQE.
2. E2: *broadcast hash join* so với *sort-merge join*, vai trò của `autoBroadcastJoinThreshold` và gợi ý `broadcast()`.
3. E3: data skew trong join và hai cách xử lý là salting và AQE skew join.
4. E4: `repartition` so với `coalesce` trước khi ghi (thời gian, số file, mức song song).
5. E5: khi nào `cache`/`persist` có lợi.

Mục 2 nhắc lại phần lý thuyết cần dùng, Mục 3 mô tả môi trường và cách đo, Mục 4 trình bày kết quả, Mục 5 trả lời tám câu hỏi checkpoint, và Mục 6 đưa ra khuyến nghị.

2 Cơ sở lý thuyết

2.1 Shuffle và số partition shuffle

Spark chia một truy vấn thành các stage, ranh giới là các *wide dependency* (phụ thuộc rộng) như `groupBy` hay `join`, nơi các bản ghi cùng khóa phải được đưa về cùng một partition [1]. Khi shuffle, phía map ghi dữ liệu đã băm theo khóa ra đĩa cục bộ, phía reduce đọc lại thành `spark.sql.shuffle.partitions` partition (mặc định 200). Mỗi partition là một task, nên con số này quyết định cả mức song song lẫn chi phí lập lịch. Quá ít partition thì task to, dễ spill hoặc thiếu bộ nhớ; quá nhiều thì có hàng trăm task tí hon mà phần chi phí khởi tạo còn lớn hơn phần việc thật [3], [4].

2.2 Adaptive Query Execution

AQE tối ưu lại kế hoạch vật lý trong lúc chạy, dựa trên thống kê thật của các stage shuffle đã xong (*map output statistics*) thay vì ước lượng từ kích thước file [3]. Bài này dùng ba tính năng của AQE: gộp các partition nhỏ liền kề tới khoảng `advisoryPartitionSizeInBytes` (nút `AQEShuffleRead coalesced`), đổi sang broadcast khi một phía thật sự nhỏ, và tách partition bị lệch trong sort-merge join (`SortMergeJoin(skew=true)`). AQE được bật mặc định từ Spark 3.2.

2.3 Chiến lược join

Spark SQL [2] chọn kiểu join theo kích thước ước lượng của hai phía. Broadcast hash join (BHJ) gom bảng nhỏ về driver rồi gửi một bản sao tới mọi executor, nên bảng lớn không phải shuffle. Sort-merge join (SMJ) shuffle cả hai phía theo khóa, sort từng partition rồi trộn. `spark.sql.autoBroadcastJoinThreshold` (mặc định 10 MB) quyết định khi nào broadcast được chọn tự động; đặt -1 sẽ tắt việc chọn tự động, nhưng gợi ý `broadcast()` viết trong mã vẫn có hiệu lực.

2.4 Data skew và salting

Skew xảy ra khi vài khóa chiếm phần lớn số bản ghi. Shuffle băm theo khóa nên mọi bản ghi của một *hot key* (khóa nóng) dồn vào một partition, sinh ra một *straggler* (task tụt hậu) trong khi các core khác ngồi chờ. Dấu hiệu dễ nhận nhất là thời gian task lớn nhất vượt xa trung vị trong cùng stage. Salting gắn thêm một khóa phụ ngẫu nhiên cho hot key và nhân bản dòng tương ứng ở bảng dimension, để hot key được rải ra nhiều partition [4], [5].

2.5 repartition, coalesce và cache

`repartition(n)` luôn shuffle để chia đều dữ liệu. `coalesce(n)` chỉ gộp các partition có sẵn qua *narrow dependency* (phụ thuộc hẹp), không shuffle, nhưng có thể kéo giảm song song của cả stage đứng trước nó [4]. `cache`/`persist` giữ kết quả trung gian trong bộ nhớ hoặc trên đĩa

để các action sau không phải tính lại cả lineage [1]. Nó chỉ có lợi khi chi phí tính lại lớn hơn chi phí materialize cộng với bộ nhớ bị chiếm.

3 Môi trường và phương pháp thực nghiệm

3.1 Môi trường

Mọi thứ chạy trong một container Docker, Spark ở chế độ local với 4 core, nên không có chi phí mạng giữa các executor (Bảng 1).

Bảng 1. Môi trường thực nghiệm.

Thành phần	Giá trị
Máy chủ	Laptop 12 luồng CPU, 32 GB RAM
Ảo hóa	Docker Desktop trên WSL2, cấp 31 GB
Spark	4.1.3 (image `spark:4.1.3-python3`), PySpark
Chế độ chạy	`local[4]` (4 core, `defaultParallelism` = 4), driver 2 GB
Thu thập số liệu	`time.time()` quanh mỗi action; Spark REST API (`/sql`, `/stages/.../taskSummary`)
Ngày thực hiện	2026-10-30

3.2 Dữ liệu

Bảng fact tổng hợp được sinh bằng `spark.range`: 60 % số dòng có khóa 0 (`HOT_SHARE = 0.6`), phần còn lại rải đều trên 200,000 khóa. Đếm lại thì được 5,998,366 dòng hot và 4,001,634 dòng thường. Các tập dữ liệu được liệt kê ở Bảng 2.

Bảng 2. Dữ liệu thực nghiệm.

Tập dữ liệu	Nguồn	Kích thước	Dùng trong
Trips	`yellow_tripdata_2024-01.parquet`	2,964,624 dòng, 47.6 MiB, 3 row group	E1, E2, E4, E5
Zones	`taxi_zone_lookup.csv`	265 dòng	E2
Fact tổng hợp	`spark.range(10,000,000)`, 60 % key = 0	10,000,000 dòng	E3

Tập dữ liệu	Nguồn	Kích thước	Dùng trong
Dim tổng hợp	`spark.range(200,000)`, cột `segment` = `seg_{key % 50}`	200,000 dòng	E3

3.3 Quy trình

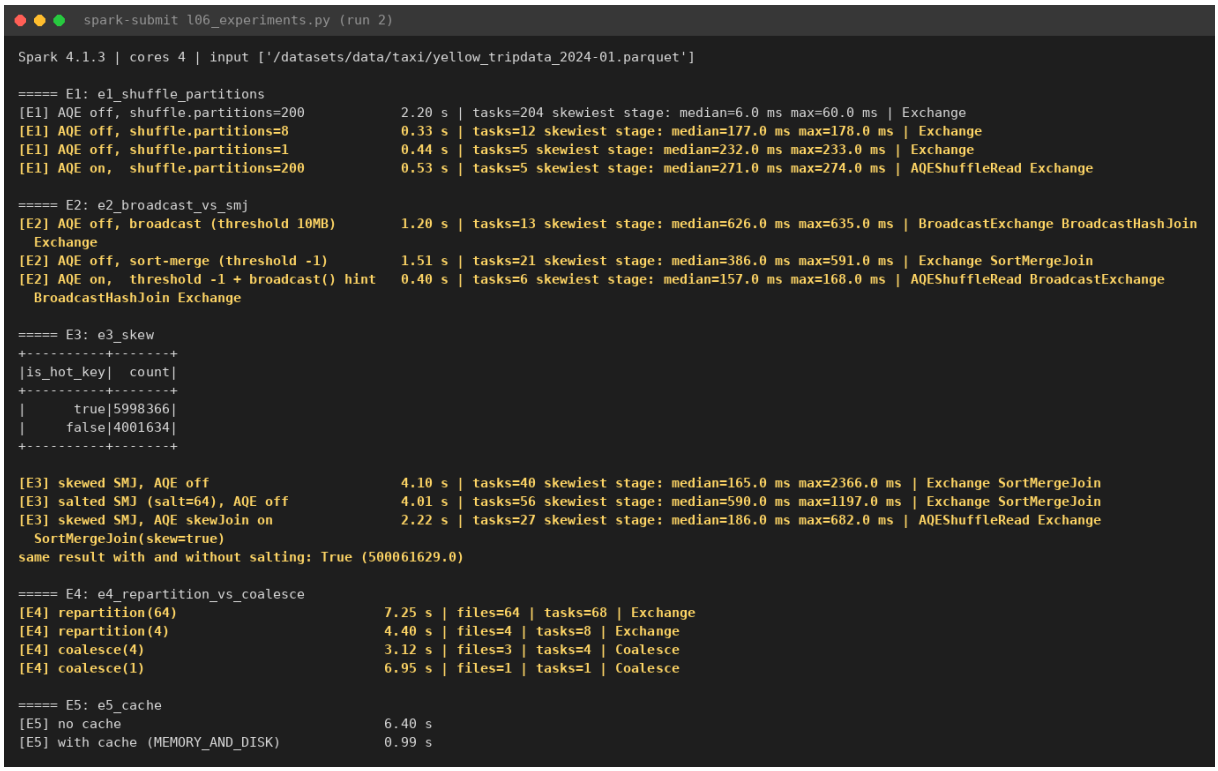
Script `l06_experiments.py` (trích đoạn ở Phụ lục B) chạy từng biến thể theo các bước sau:

1. Tạo SparkSession và chạy một `count()` để khởi động JVM, liệt kê file và làm nóng OS cache; lần này không tính.
2. Với mỗi biến thể, đặt lại cấu hình (`reset_conf`), đổi tham số cần khảo sát, rồi ép thực thi bằng `df.write.format("noop")` (tính đủ nhưng không ghi file). E4 ghi Parquet thật để đếm số file; E5 dùng `collect()` cho ba truy vấn.
3. Sau mỗi action, hàm `last_sql_summary()` gọi REST API lấy các nút của kế hoạch cuối (sau AQE), tổng số task, và median/max thời gian task tại stage lệch nhất (stage có hiệu max trừ median lớn nhất).
4. Tham số thay đổi: E1 `shuffle.partitions` $\in \{200, 8, 1\}$ khi tắt AQE và 200 khi bật AQE; E2 ngưỡng broadcast 10 MB, -1, và -1 kèm `broadcast()`; E3 SMJ 16 partition tắt AQE, sau đó salting với 64 giá trị salt, sau đó AQE skew join với `advisoryPartitionSizeInBytes=4m`, `skewedPartitionThresholdInBytes=16m`, `skewedPartitionFactor=3` và tắt coalesce; E4 bốn cách phân vùng trước khi ghi; E5 ba phép tổng hợp có và không có `persist(MEMORY_AND_DISK)`.
5. Chúng tôi chạy cả script hai lần, kết quả nối vào `results.csv` (34 dòng = 2 lần × 17 biến thể). Báo cáo dùng lần chạy thứ hai; lần đầu để xem số liệu có ổn định không. Ở lần có cờ `--hold`, Spark UI được giữ lại để chụp kế hoạch và Summary Metrics.

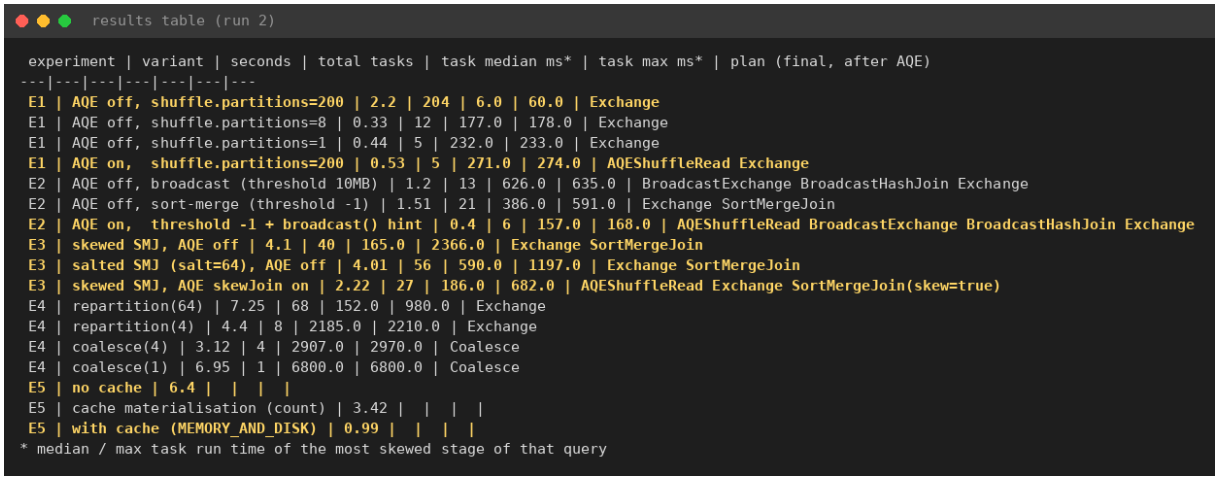
4 Kết quả thực nghiệm

4.1 Tổng quan hai lần chạy

Hình 1 là đầu ra console của lần chạy thứ hai, Hình 2 là bảng Markdown script in ra ở cuối. Toàn bộ 17 biến thể được gom lại ở Bảng 3; hai cột median/max là thời gian task ở stage lệch nhất của truy vấn. Tab SQL của phiên có giữ UI liệt kê 31 truy vấn đã xong (Hình 3), và chúng tôi dùng danh sách này để tìm kế hoạch của từng biến thể.



Hình 1. Đầu ra console của lần chạy thứ hai: thời gian, số task, median/max task và các nút kế hoạch của từng biến thể.



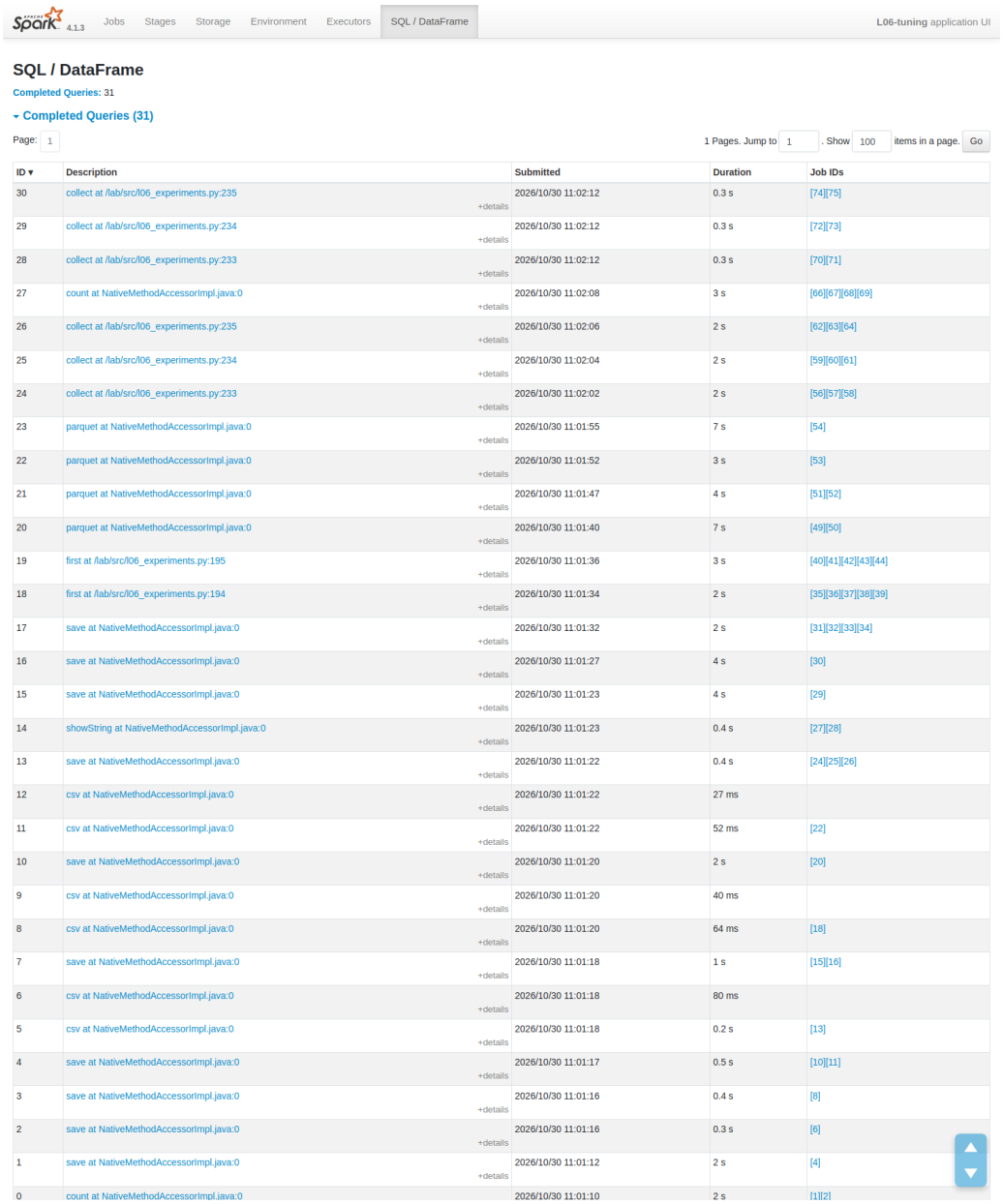
Hình 2. Bảng kết quả Markdown do script in ra ở cuối lần chạy thứ hai.

Bảng 3. Kết quả đầy đủ của lần chạy thứ hai (17 biến thể).

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=200	2.20	204	6	60	Exchange	

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E1	AQE off, shuffle.partitions=8	0.33	12	177	178	Exchange	
E1	AQE off, shuffle.partitions=1	0.44	5	232	233	Exchange	
E1	AQE on, shuffle.partitions=200	0.53	5	271	274	AQEShuffleRead Exchange	AQE gộp partition nhỏ
E2	AQE off, broadcast (threshold 10MB)	1.20	13	626	635	BroadcastExchange BroadcastHashJoin Exchange	
E2	AQE off, sort-merge (threshold -1)	1.51	21	386	591	Exchange SortMergeJoin	
E2	AQE on, threshold -1 + broadcast() hint	0.40	6	157	168	AQEShuffleRead BroadcastExchange BroadcastHashJoin Exchange	Không đủ ngưỡng broadcast
E3	skewed SMJ, AQE off	4.10	40	165	**2366**	Exchange SortMergeJoin	
E3	salted SMJ (salt=64), AQE off	4.01	56	590	1197	Exchange SortMergeJoin	cùng kết quả (500061629)
E3	skewed SMJ, AQE skewJoin on	2.22	27	186	682	AQEShuffleRead Exchange SortMergeJoin(skew=true)	partition split
E4	repartition(64)	4.25	68	152	980	Exchange	64 file
E4	repartition(4)	1.40	8	2185	2210	Exchange	4 file
E4	coalesce(4)	3.12	4	2907	2970	Coalesce	3 file
E4	coalesce(1)	6.95	1	6800	6800	Coalesce	1 file

Thí nghiệm	Biến thể	Thời gian (s)	Tổng task	Median (ms)	Max (ms)	Kế hoạch cuối	Ghi chú
E5	no cache (3 queries)	6.40					3 aggregation
E5	cache materialisation (count)	3.42					2,869,714 dòng, 59.4 MiB
E5	with cache (3 queries)	0.99					3 aggregation



Hình 3. Tab SQL / DataFrame của Spark UI với 31 truy vấn đã hoàn tất trong phiên chạy.

Lần chạy đầu (tổng 71.71 s) cho cùng xu hướng ở mọi thí nghiệm (Bảng 4). Phần lớn chênh lệch nằm trong khoảng ±20 % mà đề bài cho phép. Ngoại lệ là các biến thể rất ngắn: E2 broadcast khi tắt AQE lần đầu chỉ 0.77 s, lần hai lên 1.20 s, và E1 một partition từ 0.30 s lên 0.44 s. Với truy vấn chạy dưới một giây, vài trăm mili giây dao động đã là tỉ lệ lớn, nên chúng tôi không rút kết luận từ chênh lệch nhỏ giữa các biến thể loại này.

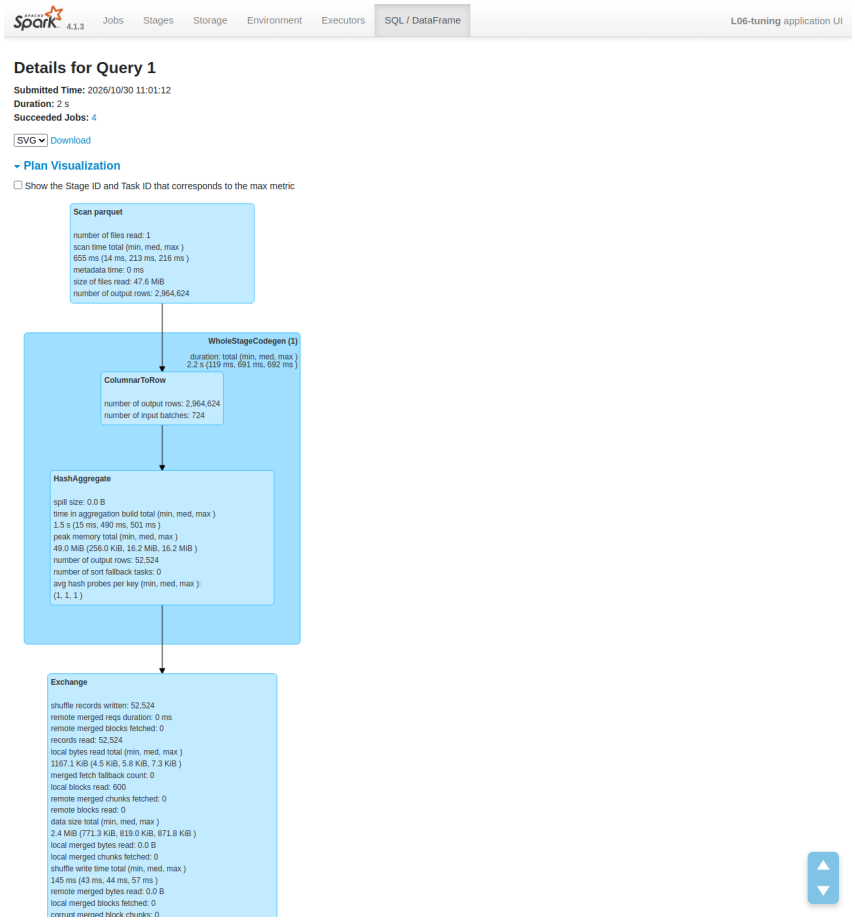
Bảng 4. So sánh một số chỉ số giữa hai lần chạy.

Chỉ số	Lần 1	Lần 2
E1, 200 partition, AQE off	2.15 s	2.20 s
E2, broadcast, AQE off	0.77 s	1.20 s

Chỉ số	Lần 1	Lần 2
E3 baseline, task max	2,686 ms	2,366 ms
E3 salting	4.28 s	4.01 s
E3 AQE skew join	2.28 s	2.22 s
E5 không cache	6.07 s	6.40 s
E5 với cache	0.88 s	0.99 s

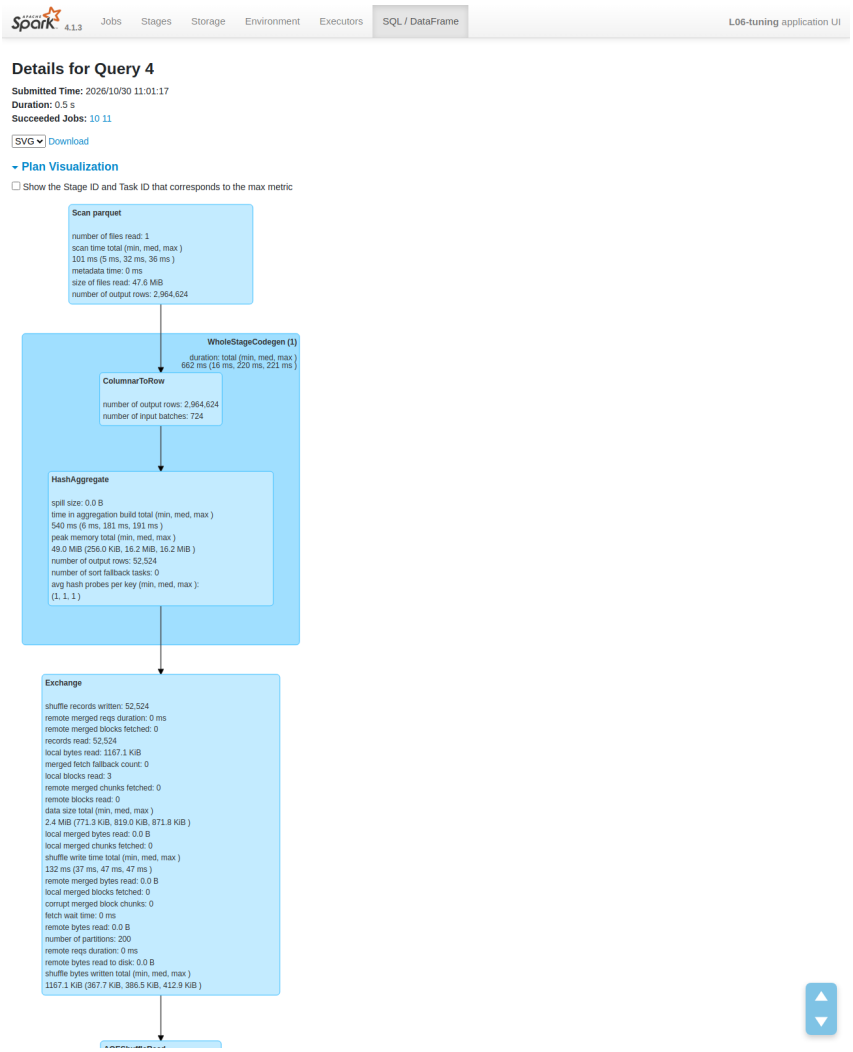
4.2 E1: số partition shuffle và AQE coalescing

Truy vấn `groupBy(PULocationID, DOLocationID).agg(count, avg)` đọc 47.6 MiB nhưng chỉ ghi khoảng 1167.1 KiB dữ liệu shuffle. Với 200 partition và AQE tắt, truy vấn sinh 204 task (4 map, 200 reduce), mỗi task reduce chạy khoảng 6 ms, và đây là biến thể chậm nhất. Kế hoạch ở Hình 4 chỉ có một `Exchange` 200 partition.



Hình 4. Kế hoạch E1 khi AQE tắt: `Exchange` với 200 partition shuffle.

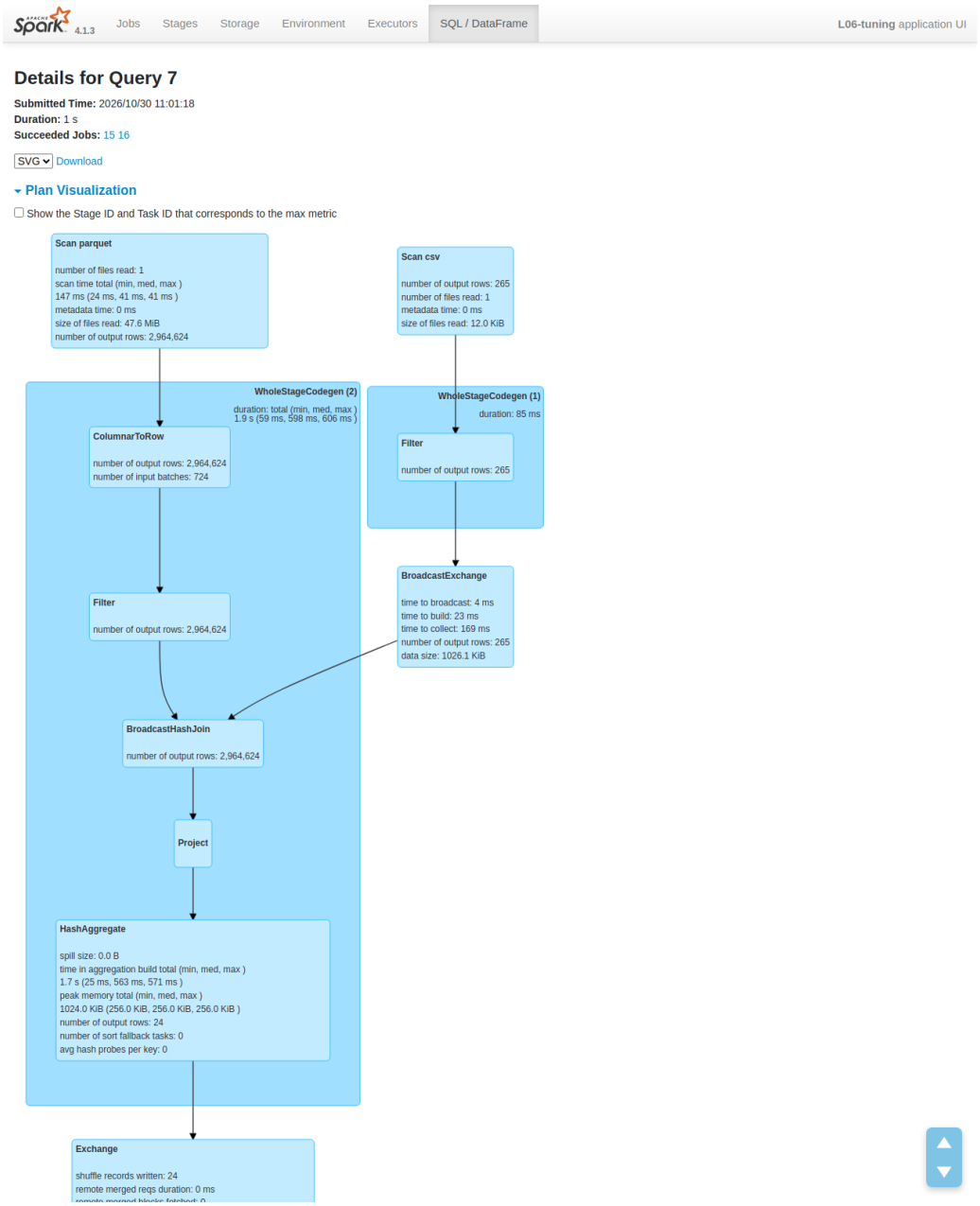
Bật AQE mà vẫn để 200 partition thì sau `Exchange` xuất hiện `AQEShuffleRead` (Hình 5), các partition nhỏ được gộp lại, tổng chỉ còn 5 task, thời gian xấp xỉ bản tinh chỉnh tay 8 partition. Bản 1 partition cũng nhanh, nhưng chỉ vì kết quả quá nhỏ (khoảng 20 nghìn nhóm); Mục 5.1 giải thích vì sao không nên dùng nó.



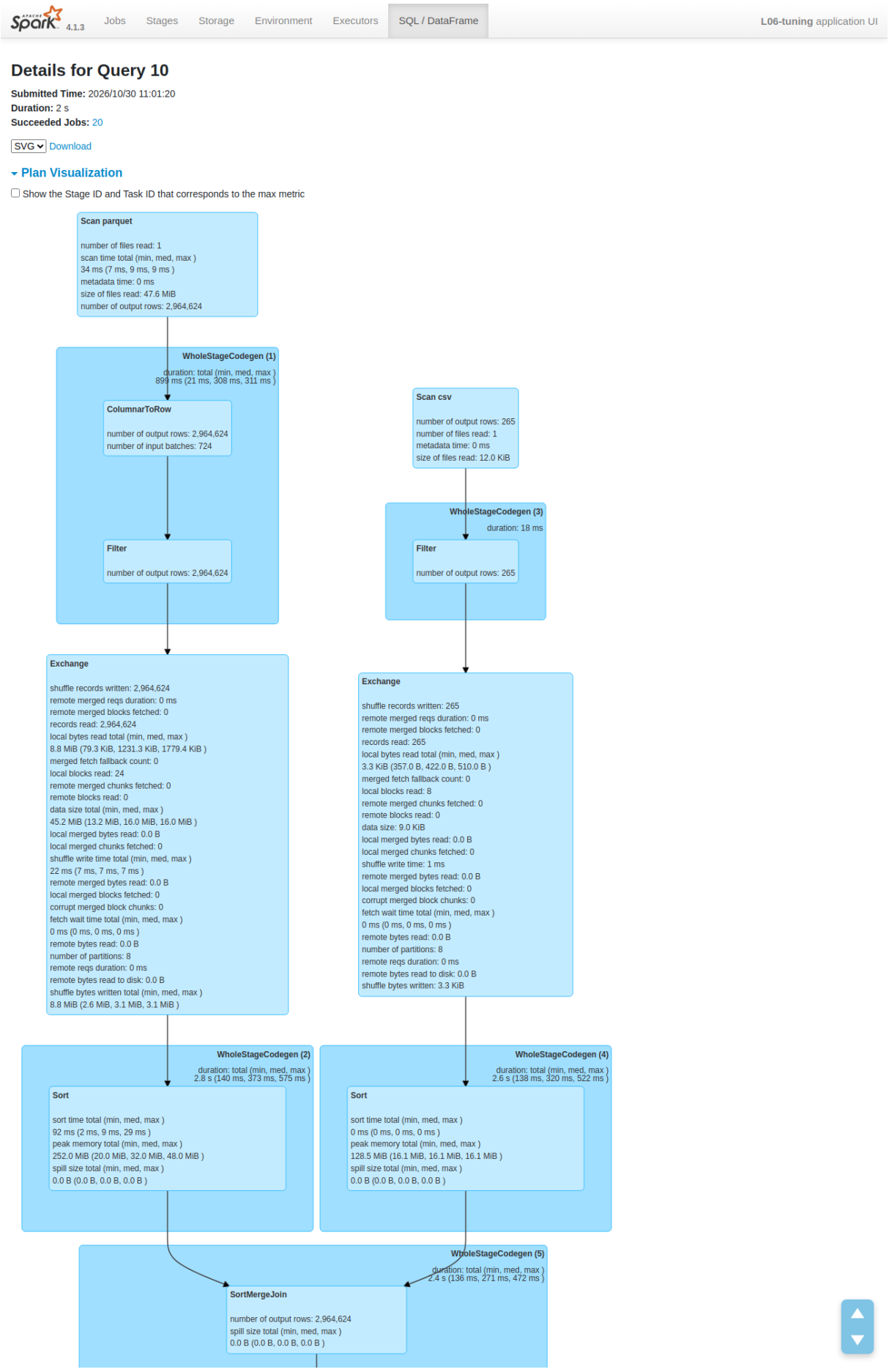
Hình 5. Kế hoạch E1 khi AQE bật: `Exchange` 200 partition theo sau là `AQEShuffleRead` ở chế độ coalesced; truy vấn đọc 2,964,624 dòng, 47.6 MiB.

4.3 E2: broadcast hash join và sort-merge join

Phép join trips ⋈ zones (zones có 265 dòng) được chạy theo ba cách. Khi ngưỡng là -1, Spark phải dùng `SortMergeJoin`: khoảng 3 triệu chuyến bị shuffle cùng bảng zone rồi sort (Hình 7). Khi broadcast, chỉ 265 dòng được gửi đi và phía trips không có `Exchange` trước join (Hình 6).



Hình 6. Kế hoạch E2 với broadcast: `BroadcastExchange` và `BroadcastHashJoin`, phía trips không có `Exchange` trước join.



Hình 7. Kế hoạch E2 với ngưỡng -1: `Exchange` và `Sort` ở cả hai phía trước `SortMergeJoin`.

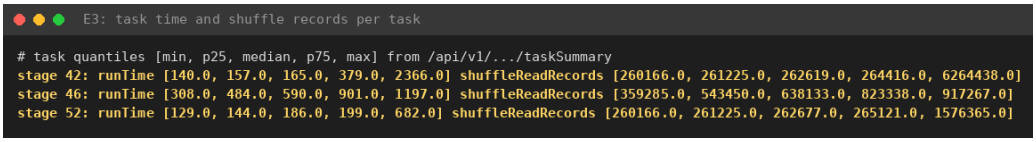
Gợi ý `F.broadcast()` vẫn có tác dụng khi ngưỡng là -1 và là biến thể nhanh nhất của E2 (0.40 s, 6 task). Chúng tôi hơi bất ngờ khi thấy broadcast tắt AQE (1.20 s) chỉ nhanh hơn SMJ (1.51 s) một chút. Lý do có lẽ là cấu hình mặc định của script chỉ dùng 8 partition nên SMJ ở đây khá nhẹ, cộng thêm dao động giữa các lần chạy đã nói ở trên. Khác biệt thật nằm ở kế hoạch: broadcast bỏ được `Exchange` và `Sort` ở phía bảng lớn, và khoản tiết kiệm này sẽ lớn dần theo kích thước dữ liệu.

4.4 E3: data skew, salting và AQE skew join

Bảng fact có 5,998,366 dòng mang khóa 0. Bằng REST API, chúng tôi lấy phân vị thời gian task và số bản ghi shuffle đọc mỗi task tại stage join của ba biến thể (Bảng 5, Hình 8).

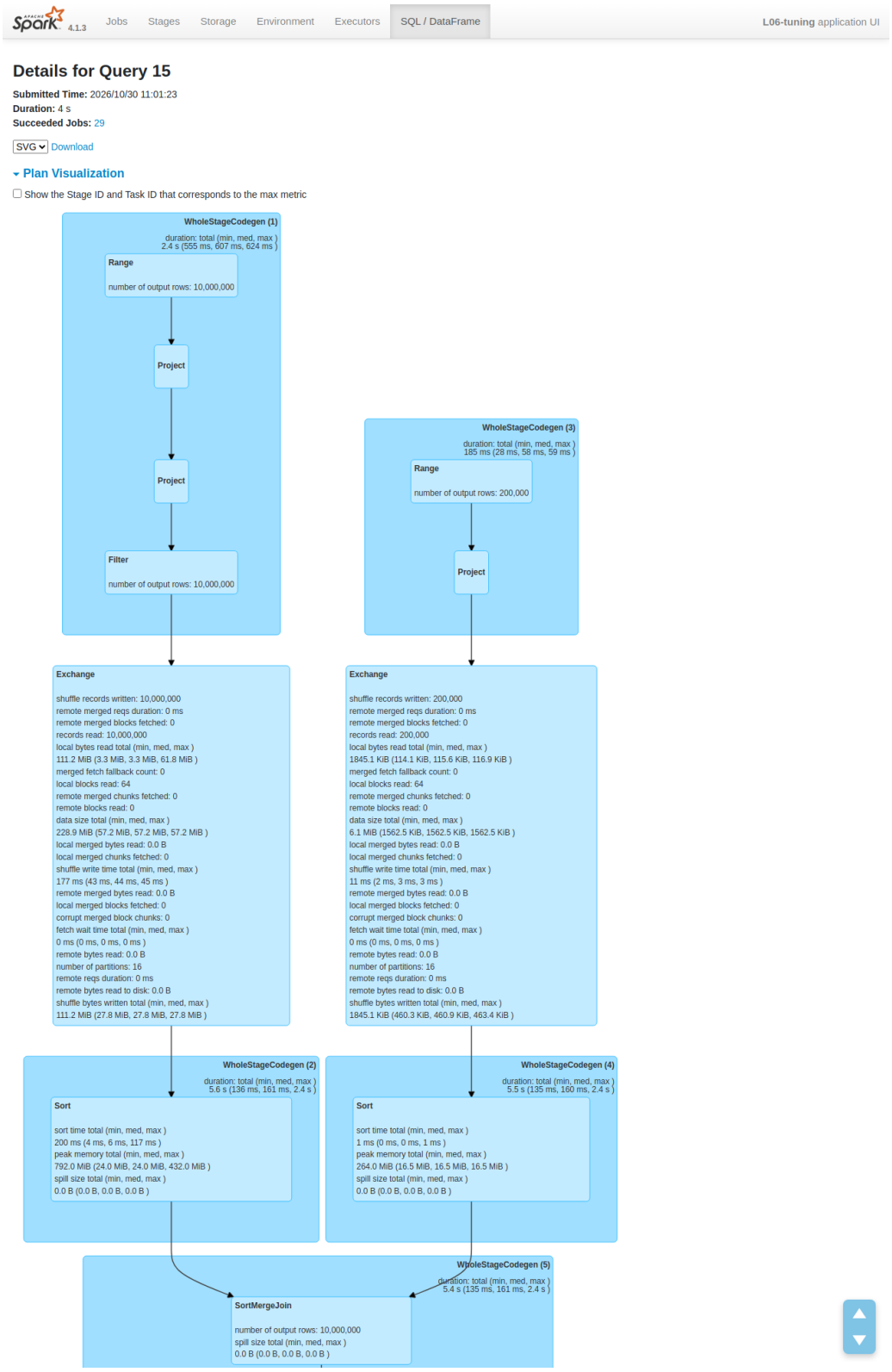
Bảng 5. Phân vị min / 25 % / median / 75 % / max tại stage join của E3 (lần chạy thứ hai).

Stage (biến thể)	Thời gian task (ms)	Bản ghi shuffle đọc / task
42: baseline, AQE off	140 / 157 / 165 / 379 / 2,366	260,166 / 261,225 / 262,619 / 264,416 / 6,264,438
46: salting 64	308 / 484 / 590 / 901 / 1,197	359,285 / 543,450 / 638,133 / 823,338 / 917,267
52: AQE skew join	129 / 144 / 186 / 199 / 682	260,166 / 261,225 / 262,677 / 265,121 / 1,576,365

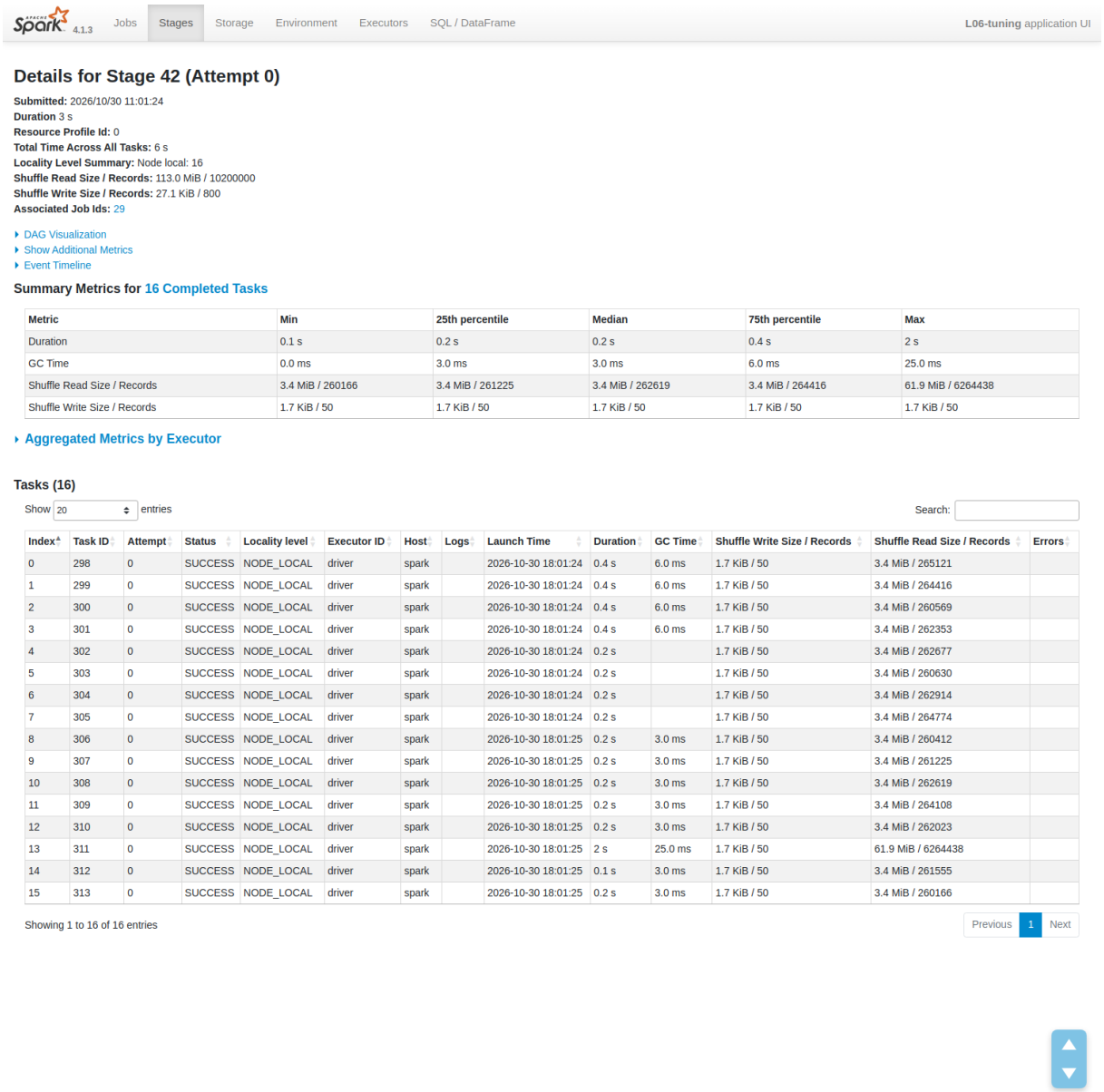


Hình 8. Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3, lấy từ Spark REST API.

Baseline. Kế hoạch là `Exchange` + `SortMergeJoin` với 16 partition (Hình 9). Ở stage 42, task chậm nhất lâu hơn median khoảng 14 lần. Theo Summary Metrics (Hình 10), task này đọc 61.9 MiB, trong khi các task khác chỉ khoảng 3.4 MiB. Cả stage đọc 113.0 MiB / 10,200,000 bản ghi qua 16 task. Trong lúc task đó chạy, ba core còn lại không có việc gì làm.

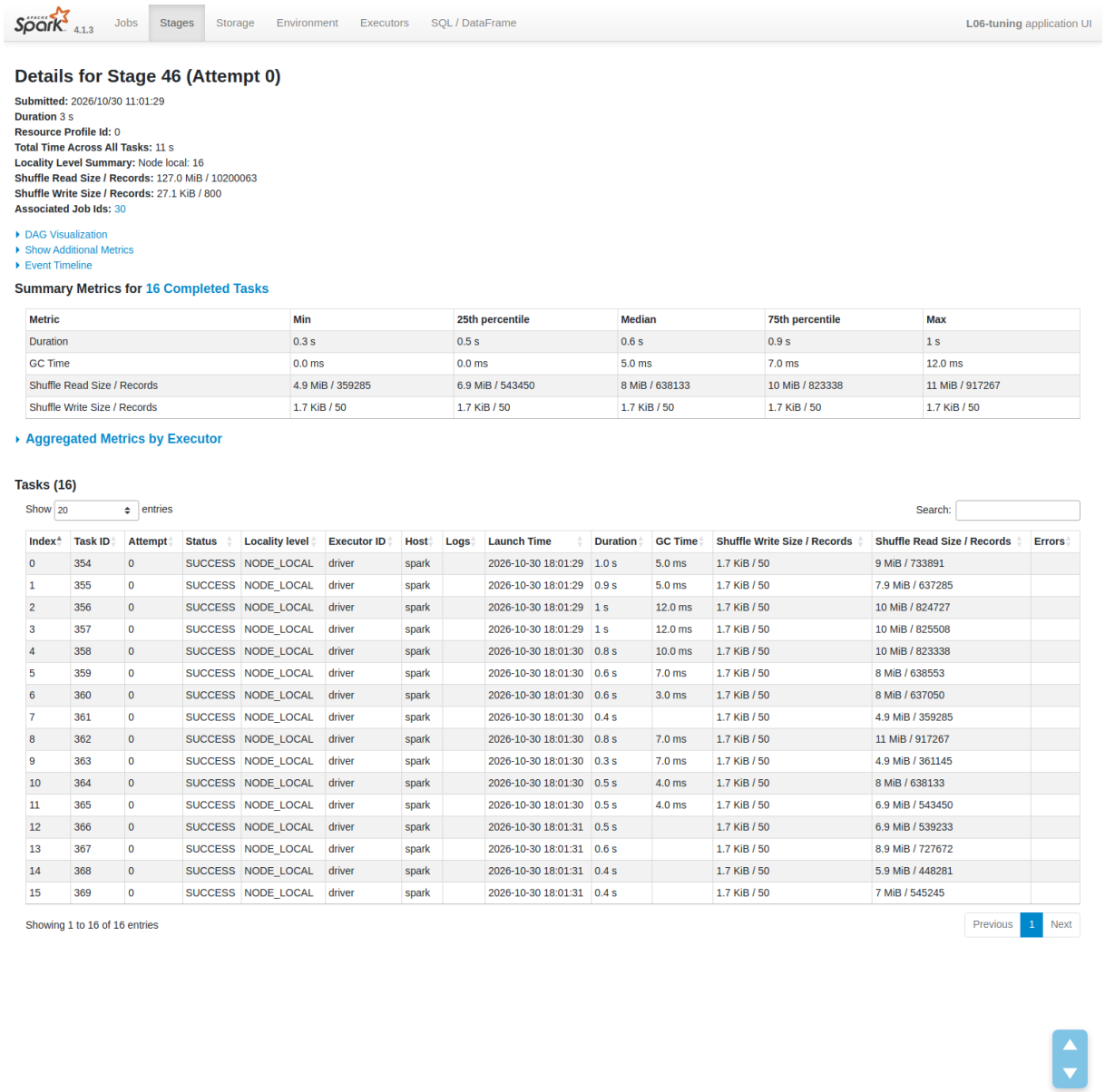


Hình 9. Kế hoạch E3 baseline (AQE tắt): sort-merge join trên dữ liệu lệch.



Hình 10. Summary Metrics của stage 42 (baseline): max 61.9 MiB / 6,264,438 bản ghi so với median 3.4 MiB / 262,619 bản ghi; task index 13 mất 2 s.

Salting. Rải hot key ra 64 giá trị salt làm task max giảm gần một nửa và số bản ghi lớn nhất mỗi task chỉ còn chưa tới một triệu (Hình 11). Bù lại, median tăng lên 590 ms vì partition nào cũng nhận thêm một phần hot key, lại thêm chi phí `rand()`, cột join phụ và 64 bản sao dòng dim. Kết quả là thời gian tổng gần như đứng yên (4.01 s so với 4.10 s). Tổng `amount` vẫn đúng (500061629), nghĩa là salting không làm sai kết quả.



Spark 4.1.3 Jobs Stages Storage Environment Executors SQL / DataFrame L06-tuning application UI

Details for Stage 46 (Attempt 0)

Submitted: 2026/10/30 11:01:29
Duration 3 s
Resource Profile Id: 0
Total Time Across All Tasks: 11 s
Locality Level Summary: Node local: 16
Shuffle Read Size / Records: 127.0 MiB / 10200063
Shuffle Write Size / Records: 27.1 KiB / 800
Associated Job Ids: 30

▸ DAG Visualization
▸ Show Additional Metrics
▸ Event Timeline

Summary Metrics for 16 Completed Tasks

Metric	Min	25th percentile	Median	75th percentile	Max
Duration	0.3 s	0.5 s	0.6 s	0.9 s	1 s
GC Time	0.0 ms	0.0 ms	5.0 ms	7.0 ms	12.0 ms
Shuffle Read Size / Records	4.9 MiB / 359285	6.9 MiB / 543450	8 MiB / 638133	10 MiB / 823338	11 MiB / 917267
Shuffle Write Size / Records	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50	1.7 KiB / 50

▸ Aggregated Metrics by Executor

Tasks (16)

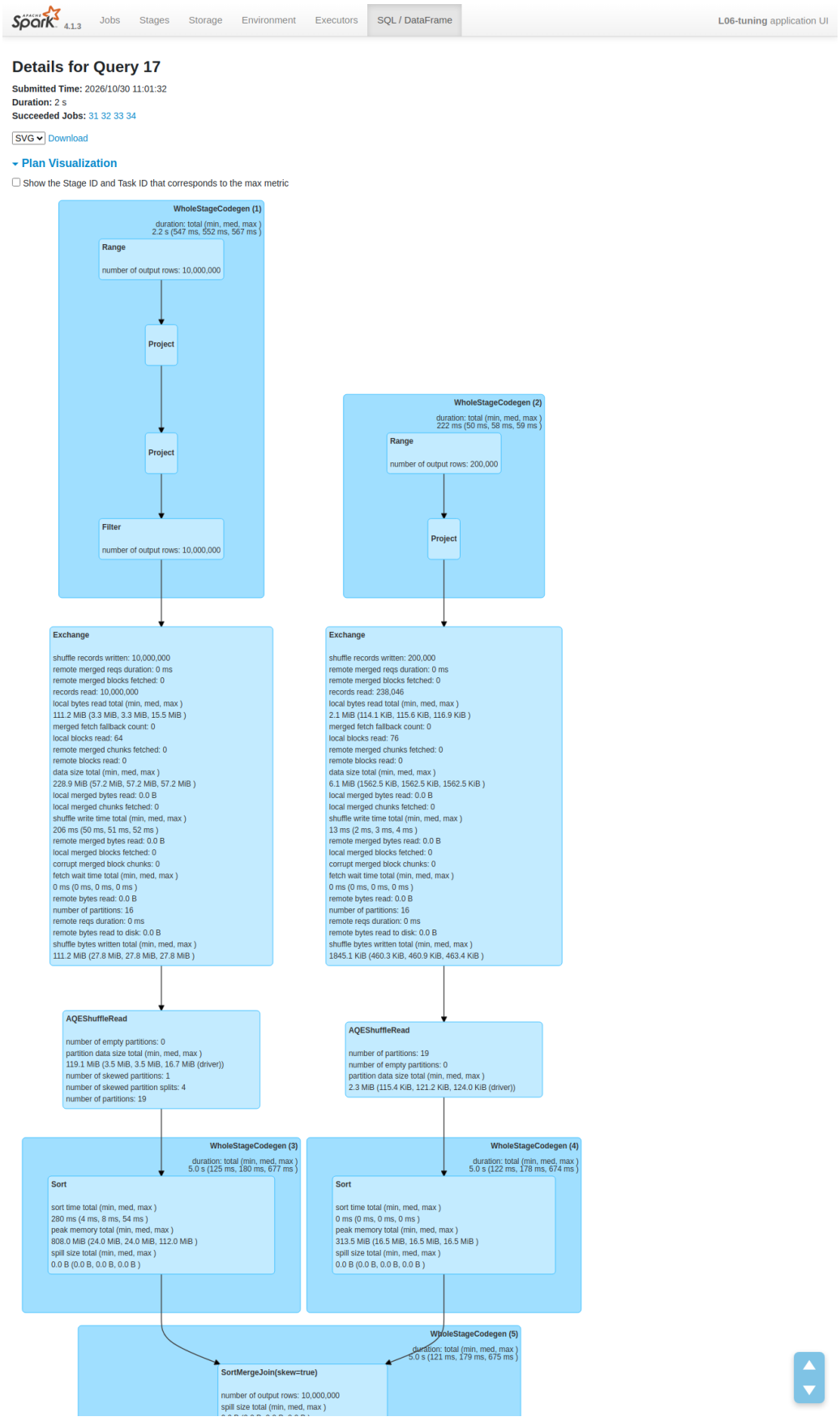
Show 20 entries Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Shuffle Write Size / Records	Shuffle Read Size / Records	Errors
0	354	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1.0 s	5.0 ms	1.7 KiB / 50	9 MiB / 733891	
1	355	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	0.9 s	5.0 ms	1.7 KiB / 50	7.9 MiB / 637285	
2	356	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 824727	
3	357	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:29	1 s	12.0 ms	1.7 KiB / 50	10 MiB / 825508	
4	358	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	10.0 ms	1.7 KiB / 50	10 MiB / 823338	
5	359	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	7.0 ms	1.7 KiB / 50	8 MiB / 638553	
6	360	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.6 s	3.0 ms	1.7 KiB / 50	8 MiB / 637050	
7	361	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.4 s		1.7 KiB / 50	4.9 MiB / 359285	
8	362	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.8 s	7.0 ms	1.7 KiB / 50	11 MiB / 917267	
9	363	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.3 s	7.0 ms	1.7 KiB / 50	4.9 MiB / 361145	
10	364	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	8 MiB / 638133	
11	365	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:30	0.5 s	4.0 ms	1.7 KiB / 50	6.9 MiB / 543450	
12	366	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.5 s		1.7 KiB / 50	6.9 MiB / 539233	
13	367	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.6 s		1.7 KiB / 50	8.9 MiB / 727672	
14	368	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	5.9 MiB / 448281	
15	369	0	SUCCESS	NODE_LOCAL	driver	spark		2026-10-30 18:01:31	0.4 s		1.7 KiB / 50	7 MiB / 545245	

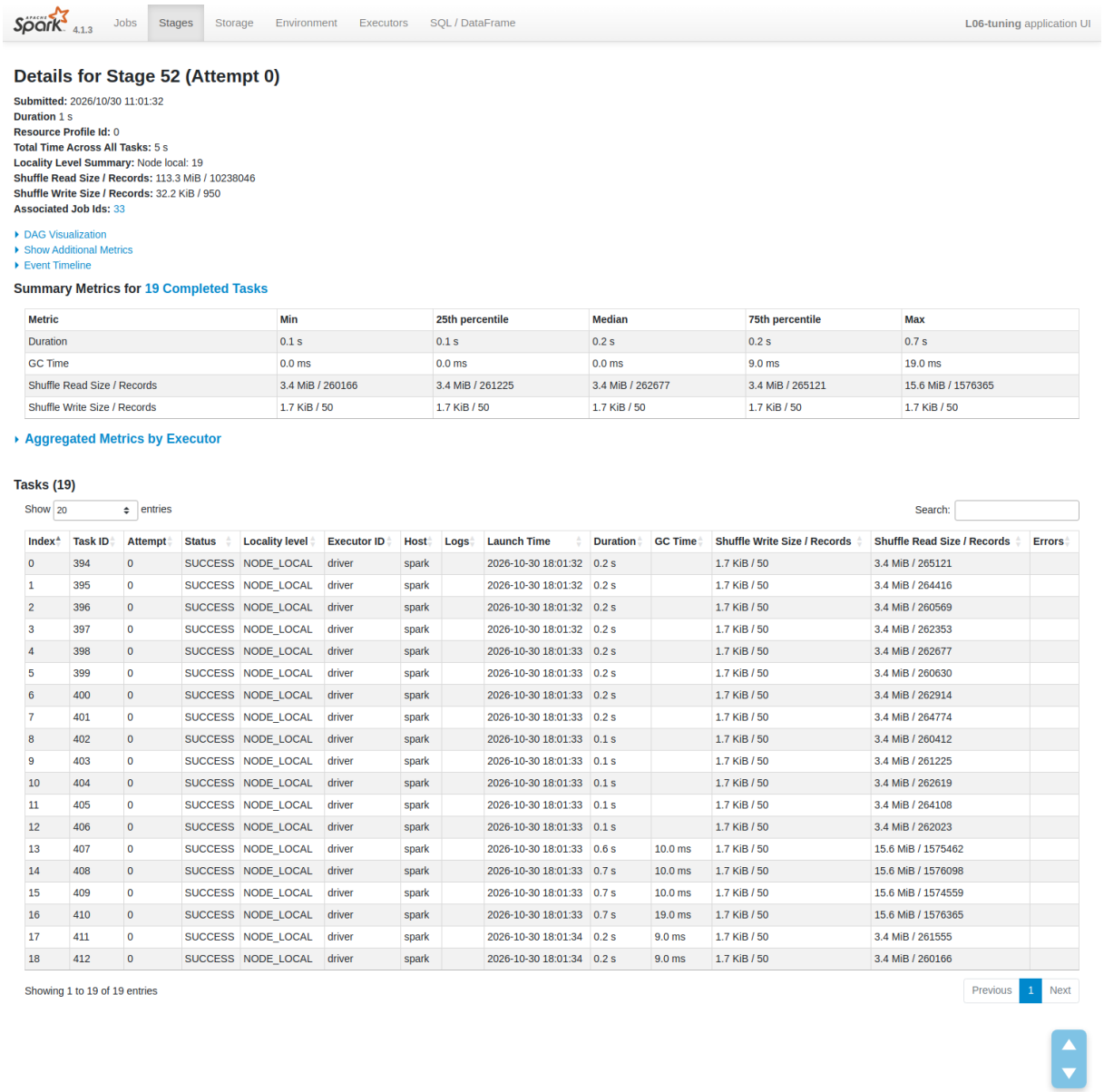
Showing 1 to 16 of 16 entries Previous 1 Next

Hình 11. Summary Metrics của stage 46 (salting 64): phân bố bản ghi đều hơn, max 917,267 bản ghi, nhưng median thời gian task tăng lên 590 ms.

AQE skew join. Kế hoạch cuối có `SortMergeJoin(skew=true)`, và `AQEShuffleRead` báo *number of skewed partitions: 1, skewed partition splits: 4, number of partitions: 19* (Hình 12). Phía fact ghi 10,000,000 bản ghi shuffle (111.2 MiB); sau khi đọc, partition có median 3.5 MiB và max 16.7 MiB. Ở stage 52 (Hình 13), số bản ghi lớn nhất mỗi task xấp xỉ một phần tư của baseline, đúng với 4 split. Thời gian tổng còn **2.22 s, nhanh gần gấp đôi baseline**, và không phải sửa dòng truy vấn nào. Lúc đầu chúng tôi để ngưỡng mặc định thì AQE không tách gì, vì partition nóng chỉ có 61.9 MiB, thấp hơn nhiều so với mức 256 MB mặc định. Vì vậy phải hạ ngưỡng như đã nêu ở Mục 3.



Hình 12. Kế hoạch E3 với AQE skew join: `AQEShuffleRead` báo 1 partition lệch được tách thành 4 split (16 thành 19 partition), sau đó là `SortMergeJoin(skew=true)`.



Hình 13. Summary Metrics của stage 52 (AQE skew join): max 682 ms và 1,576,365 bản ghi mỗi task.

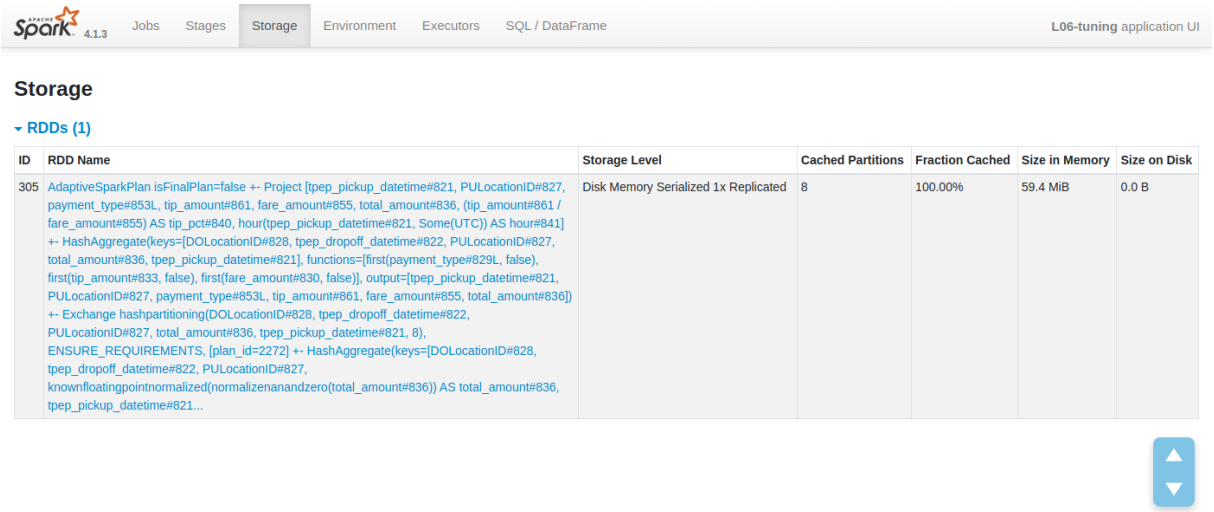
4.5 E4: repartition và coalesce trước khi ghi

`repartition(64)` chậm nhất vì vừa có một shuffle đầy đủ vừa ghi 64 file nhỏ. `coalesce(4)` không shuffle nên nhanh nhất, nhưng chỉ ra 3 file chứ không phải 4. Ban đầu chúng tôi nghĩ có lỗi; xem lại thì file Parquet đầu vào chỉ có 3 row group nên một trong 4 partition đầu vào rỗng. `coalesce(1)` dồn cả bước đọc lẫn bước ghi vào một task duy nhất và mất hết song song. `repartition(4)` cho 4 file đều nhau, đổi lại phải trả thêm một shuffle (Bảng 3).

4.6 E5: cache

DataFrame "cleaned" gồm một phép lọc và `dropDuplicates` trên năm cột, tức là shuffle toàn bộ khoảng 3 triệu dòng. Nó được dùng cho ba phép tổng hợp: theo giờ, theo `PULocationID` và theo

`payment_type`. Không cache thì dedup chạy lại ba lần. Có cache thì `count()` để materialize mất 3.42 s và trả về 2,869,714 dòng, sau đó ba truy vấn chỉ còn **0.99 s**. Tab Storage (Hình 14) cho thấy 8/8 partition đã được cache (100 %), chiếm 59.4 MiB bộ nhớ và 0.0 B đĩa, mức `Disk Memory Serialized 1x Replicated`; REST API báo `memoryUsed` = 62,240,847 byte. Tính cả thời gian materialize, cache vẫn lời (khoảng 4.4 s so với 6.4 s), và càng dùng lại nhiều lần thì càng lời.



Hình 14. Tab Storage của Spark UI: DataFrame `cleaned` được cache 8/8 partition, 100 %, 59.4 MiB trong bộ nhớ.

4.7 So sánh với kết quả tham chiếu

Đề bài kèm một bảng tham chiếu đo trên máy Apple M-series, Docker 4 GB, `local[4]` (Bảng 6). Thời gian trên máy chúng tôi cao hơn khoảng 1.3 đến 1.9 lần, có thể do khác phần cứng và có thêm lớp WSL2. Mọi xu hướng thì khớp: trong E1, 200 partition chậm nhất và AQE bắt kịp bản chỉnh tay; trong E2, gợi ý broadcast nhanh nhất; baseline E3 có tỉ lệ max/median lớn, salting không rút được thời gian tổng còn AQE skew join nhanh nhất; và `coalesce(4)` cũng ra 3 file. Chỗ khác là số task của AQE skew join (27 so với 43), vì số partition sau khi tách phụ thuộc kích thước partition thực tế lúc chạy.

Bảng 6. So sánh với bảng kết quả tham chiếu của bài lab.

Chỉ số	Tham chiếu	Lần 2
E1, 200 partition, AQE off	1.29 s	2.20 s
E1, AQE on	0.21 s	0.53 s
E2, broadcast hint	0.24 s	0.40 s
E3 baseline (median / max task)	110 / 1,263 ms	165 / 2,366 ms
E3 salting	2.97 s	4.01 s

Chỉ số	Tham chiếu	Lần 2
E3 AQE skew join (thời gian, task)	1.73 s, 43 task	2.22 s, 27 task
E4 coalesce(4)	1.85 s, 3 file	3.12 s, 3 file
E5 không cache / có cache	4.11 / 0.70 s	6.40 / 0.99 s

5 Thảo luận

5.1 Vì sao 200 partition chậm, và vì sao không nên đặt 1 partition dù nó chạy nhanh?

Kết quả `groupBy(PU, DO)` chỉ có khoảng 20 nghìn nhóm, dữ liệu shuffle chưa tới 1.2 MiB. Chia nhỏ chừng đó ra 200 phần thì mỗi task reduce chỉ làm việc khoảng 6 ms, còn chi phí lập lịch, serialize và khởi tạo của 204 task lớn hơn phần việc thật. Vì vậy bản này chậm hơn bản 8 partition khoảng 6.7 lần (Bảng 3). Một partition chạy nhanh ở đây chỉ vì shuffle quá nhỏ. Với dữ liệu thật, một reducer phải ôm toàn bộ dữ liệu shuffle: không có song song, dễ spill ra đĩa và dễ OOM. Thực tế nên nhắm khoảng 100–200 MB mỗi partition shuffle, hoặc để AQE tự quyết.

5.2 `AQEShuffleRead coalesced` làm gì, và AQE biết gì mà planner tĩnh không biết?

Khi stage map của shuffle chạy xong, AQE có kích thước thật của từng partition shuffle. `AQEShuffleRead coalesced` cho một reducer đọc gộp nhiều partition nhỏ liền kề cho tới khi đạt `advisoryPartitionSizeInBytes`. Trong E1, 200 partition chỉ còn một task reduce và tổng số task giảm từ 204 xuống 5 (Hình 5). Planner tĩnh chỉ có ước lượng dựa trên kích thước file, và ước lượng này hay sai sau filter, join hoặc aggregate.

5.3 Con số nào cho thấy skew, vì sao salting không rút ngắn thời gian tổng trên 4 core, và khi nào salting thắng?

Bằng chứng là chênh lệch giữa task max và median ở stage join: **2,366 ms so với 165 ms**, và task chậm nhất đọc 6,264,438 bản ghi trong khi median là 262,619 (Bảng 5, Hình 10).

Salting cắt task max xuống 1,197 ms nhưng thời gian tổng gần như không đổi. Một phần là vì salting thêm việc ở mọi chỗ: `rand()` cho từng dòng, thêm cột join, nhân dòng dim lên 64 lần. Phần lớn hơn là khoảng 6 triệu dòng hot vẫn phải chia trên 16 partition chạy trên 4 core. Tổng khối lượng việc không giảm, chỉ được chia lại, nên median tăng từ 165 lên 590 ms. Lần chạy đầu cũng vậy: salting chỉ nhanh hơn baseline một chút (4.28 s so với 4.62 s), còn bảng tham chiếu thậm chí ghi salting chậm hơn (2.97 s so với 2.45 s). Salting sẽ thắng rõ khi có nhiều core và nhiều partition, vì lúc đó một straggler giữ hàng trăm executor rảnh. Nó cũng có lợi khi mỗi

dòng xử lý nặng (dòng rộng, join many-to-many, có spill) hoặc khi skew nặng hơn, ví dụ 90 % hot.

5.4 AQE nhận ra partition lệch thế nào, xử lý ra sao, và kém salting ở đâu?

Một partition bị coi là lệch khi nó vừa lớn hơn `skewedPartitionFactor` lần median, vừa lớn hơn `skewedPartitionThresholdInBytes`. Mặc định là 5 lần và 256 MB; bài lab hạ xuống 3 lần và 16 MB cho vừa dữ liệu nhỏ. Partition hot ở đây là 61.9 MiB so với median 3.4 MiB, thỏa cả hai điều kiện. AQE tách partition đó theo các khoảng map output (4 split, nên 16 partition thành 19), rồi nhân bản partition tương ứng ở phía còn lại; kế hoạch hiện `SortMergeJoin(skew=true)` (Hình 12). Số bản ghi lớn nhất mỗi task còn 1,576,365, xấp xỉ 6,264,438 / 4.

AQE có mấy giới hạn mà salting không có. Nó chỉ áp dụng cho join có shuffle ở cả hai phía (sort-merge hoặc shuffled hash join), và không tách được phía phải giữ nguyên của outer join. Ngưỡng phải hợp với dữ liệu, nếu không nó sẽ không kích hoạt; chúng tôi đã gặp đúng chuyện này ở E3. Quan trọng nhất là AQE không xử lý skew trong `groupBy` hay skew phát sinh sau join, trong khi salting hoặc aggregate hai pha làm được.

5.5 Khi nào không nên broadcast dù một phía "nhỏ"?

Không nên broadcast trong các trường hợp sau:

- bảng vẫn lớn khi nằm trong bộ nhớ: driver phải collect rồi gửi bản sao tới mọi executor, dễ OOM ở driver hoặc broadcast timeout;
- thống kê đánh giá thấp kích thước thật, ví dụ sau filter hoặc UDF;
- join là full outer join (không hỗ trợ), hoặc phía cần broadcast lại là phía được giữ của outer join;
- join chạy lặp lại trên dữ liệu thay đổi, nên mỗi lần phải dựng lại broadcast.

Bảng zones trong E2 chỉ có 265 dòng và không rơi vào trường hợp nào ở trên, nên broadcast là đúng (Hình 6).

5.6 `repartition(n)` hay `coalesce(n)`: cái nào shuffle, cái nào tạo file lệch, cái nào làm giảm song song stage trước?

`repartition(n)` luôn shuffle. Nhờ vậy partition cân bằng (4 file đều nhau) và stage trước vẫn giữ mức song song, cái giá là thêm một shuffle. `coalesce(n)` gộp partition có sẵn mà không shuffle nên rẻ hơn, nhưng file ra có thể lệch: ở E4 chỉ có 3 file vì đầu vào chỉ có 3 row group. Vì `coalesce` là narrow dependency, nó kéo luôn stage trước về cùng số task. `coalesce(1)` đọc và ghi trong một task duy nhất, mất 6,800 ms (Bảng 3).

5.7 Vì sao cache có ích trong E5, còn cache một DataFrame rẻ thì thường không?

DataFrame được cache trong E5 là kết quả của một bước đắt (`dropDuplicates`, shuffle toàn bộ khoảng 3 triệu dòng) và được dùng lại ba lần. Cache nên bước này chỉ chạy một lần thay vì ba (Hình 14). Một phép quét Parquet chỉ lấy vài cột thì vốn đã rẻ nhờ đọc vectorized, column pruning và predicate pushdown; cache nó chỉ tốn thêm bộ nhớ và thời gian materialize. Nguyên tắc là cache cái gì vừa đắt vừa được dùng lại, xong thì gọi `unpersist()`.

5.8 Nên tinh chỉnh gì trước khi thêm phần cứng?

1. Đọc kế hoạch và Spark UI: tìm stage chậm nhất, spill, thời gian GC, và tỉ lệ task max/median để phát hiện skew (như Bảng 5).
2. Lọc và chọn cột càng sớm càng tốt: partition pruning, predicate/column pushdown, dùng định dạng cột (Parquet/ORC).
3. Để partition shuffle có kích thước hợp lý: bật AQE (coalesce và skew join), nhắm khoảng 100–200 MB mỗi partition.
4. Chọn đúng kiểu join: broadcast bảng nhỏ; nếu có skew thì dùng AQE skew join, salting, hoặc tách hot key.
5. Tránh UDF Python (dùng hàm built-in hoặc pandas UDF), chỉ cache khi cần, và tránh sinh file nhỏ khi ghi (repartition hoặc compaction).

5.9 Hạn chế của thực nghiệm

Mọi thứ chạy local trên một máy 4 core, không có chi phí mạng, nên chênh lệch giữa các chiến lược nhỏ hơn nhiều so với trên cluster. Vì thế cột max/median đáng tin hơn thời gian tuyệt đối. Dữ liệu nhỏ (một tháng taxi, 10 triệu dòng tổng hợp) buộc phải hạ ngưỡng skew của AQE. Chỉ có hai lần chạy, và các biến thể dưới 1 s dao động khá mạnh (Bảng 4). Lớp Docker/WSL2 cũng làm I/O chậm hơn so với máy tham chiếu. Các phần mở rộng (25 triệu dòng, `HOT_SHARE = 0.9`, bucketing, shuffled hash join) chưa được làm.

6 Kết luận

Ngay trên một máy, các tham số tinh chỉnh đã cho khác biệt đo được. Số partition shuffle không hợp làm truy vấn chậm khoảng 6.7 lần, và AQE tự sửa được việc này. Skew lộ ra rõ qua tỉ lệ task max/median khoảng 14 lần. AQE skew join giảm gần một nửa thời gian tổng mà không phải sửa mã, còn salting chỉ chia lại việc khi số core có hạn. `coalesce` rẻ nhưng có thể sinh file lệch và kéo stage trước về một task. Cache chỉ có lợi khi kết quả đắt và được dùng lại.

Với một job production trên cluster 20 node, chúng tôi đề xuất:

1. Luôn bật AQE (mặc định từ Spark 3.2 và trong Spark 4). Đặt `shuffle.partitions` ban

đầu cao, khoảng 2–3 lần tổng số core (400–800), và để `coalescePartitions` gộp về khoảng 64–128 MB mỗi partition (`advisoryPartitionSizeInBytes`). Không đặt cứng 1 hay 8.

2. Broadcast các bảng dimension nhỏ. Có thể nâng `autoBroadcastJoinThreshold` lên khoảng 50–100 MB nếu executor đủ bộ nhớ, và dùng `broadcast()` khi thống kê sai. Không broadcast bảng vài trăm MB vì driver phải collect trước, dễ OOM.
3. Với skew, để AQE skew join chạy với ngưỡng mặc định (5 lần, 256 MB). Với hot key đã biết, hoặc skew trong `groupBy`/outer join mà AQE không xử lý được, dùng salting hoặc tách hot key ra xử lý riêng. Theo dõi tỉ lệ task max/median trong UI.
4. Khi ghi output, dùng `repartition(n)` hoặc `repartition(col)` để mỗi file khoảng 128 MB–1 GB, tránh sinh hàng nghìn file nhỏ. Tránh `coalesce(1)` trên dữ liệu lớn.
5. Chỉ `persist(MEMORY_AND_DISK)` những kết quả đắt và được dùng lại ít nhất hai lần, rồi `unpersist()` ngay. Không cache phép quét Parquet rẻ.

Nếu có thêm thời gian, chúng tôi muốn chạy lại E3 với 25 triệu dòng và 90 % hot để xem ba chiến lược thay đổi thế nào khi dữ liệu lớn lên, thử shuffled hash join với dimension lớn, dùng bucketing để bỏ `Exchange`, và lặp lại toàn bộ trên một cluster nhiều node thật.

Tài liệu tham khảo

[1] M. Zaharia *et al.*, “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

[2] M. Armbrust *et al.*, “Spark SQL: Relational Data Processing in Spark,” in *Proc. ACM SIGMOD International Conference on Management of Data*, 2015.

[3] The Apache Software Foundation, “Performance Tuning – Spark SQL, DataFrames and Datasets,” Apache Spark Documentation. https://spark.apache.org/docs/latest/sql-performance-tuning.html

[4] B. Chambers and M. Zaharia, *Spark: The Definitive Guide*. Sebastopol, CA: O’Reilly Media, 2018.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O’Reilly Media, 2017.

[6] The Apache Software Foundation, “Monitoring and Instrumentation (REST API),” Apache Spark Documentation. https://spark.apache.org/docs/latest/monitoring.html

[7] J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters,” in *Proc. 6th Symposium on Operating Systems Design and Implementation (OSDI)*, 2004.

Phụ lục A: Danh mục tệp nộp kèm

Các tệp nộp kèm báo cáo được liệt kê ở Bảng 7.

Bảng 7. Danh mục tệp nộp kèm.

Tệp	Nội dung
`l06_experiments.py`	Script thí nghiệm: E1 bật AQE, E2 gợi ý `broadcast()`, E3 salting và AQE skew join
`results.csv`	34 dòng (2 lần chạy × 17 biến thể)
`RESULTS.md`	Bảng lần chạy thứ hai, nhận xét E1–E5, khuyến nghị cho cluster 20 node
`01_run1.txt`, `02_run2.txt`	Log console của hai lần chạy
`03_e3_task_quantiles.txt`	Phân vị thời gian task và số bản ghi shuffle mỗi task của ba stage join E3 (REST API)
`img/`	Ảnh chụp console và Spark UI

Phụ lục B: Trích đoạn mã nguồn

Trong `l06_experiments.py`, ở E1 chúng tôi bật lại AQE nhưng giữ 200 partition để xem AQE tự gộp:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__shuffle__partitions=200)
run("E1", "AQE on,  shuffle.partitions=200", q(), "AQE coalesces small
 ↪  shuffle partitions")
```

Ở E2, ngưỡng được để ở -1 nhưng gắn gợi ý broadcast cho bảng zones:

```
set_conf(spark__sql__adaptive__enabled=True,
 ↪  spark__sql__autoBroadcastJoinThreshold=-1)
run("E2", "AQE on,  threshold -1 + broadcast() hint",
    trips_df().join(F.broadcast(zones_df()), F.col("PULocationID") ==
 ↪  F.col("LocationID"))
    .groupBy("Borough").agg(F.count("*").alias("trips")),
    "an explicit hint beats the threshold")
```

Salting ở E3 chỉ gắn salt cho hot key; phía dim nhân dòng hot lên `SALT` = 64 bản:

```
hot = F.col("key") == 0
fact_s = fact.withColumn(
    "salt", F.when(hot, (F.rand(seed=5) *
     ↪  SALT).cast("int")).otherwise(F.lit(0)))
```

```
salts = spark.range(SALT).select(F.col("id").cast("int").alias("salt"))
dim_s = (dim.filter(~hot).withColumn("salt", F.lit(0))
         .unionByName(dim.filter(hot).crossJoin(salts)))
salted = (fact_s.join(dim_s, ["key", "salt"])
          .groupBy("segment").agg(F.sum("amount").alias("amount")))
run("E3", f"salted SMJ (salt={SALT}), AQE off", salted)
```

Biến thể cuối của E3 bật AQE skew join và hạ ngưỡng cho vừa dữ liệu cỡ lab, đồng thời tắt coalesce để thấy rõ phần tách:

```
set_conf(spark__sql__adaptive__enabled=True,
         spark__sql__adaptive__skewJoin__enabled=True,
         spark__sql__adaptive__coalescePartitions__enabled=False,
         spark__sql__adaptive__advisoryPartitionSizeInBytes="4m",
 ↪  spark__sql__adaptive__skewJoin__skewedPartitionThresholdInBytes="16m",
         spark__sql__adaptive__skewJoin__skewedPartitionFactor=3)
run("E3", "skewed SMJ, AQE skewJoin on",
    fact.join(dim, "key").groupBy("segment").agg(F.sum("amount")),
    "look for AQEShuffleRead (skewed) in the SQL tab")
```